

Agent-Based Ecosystem

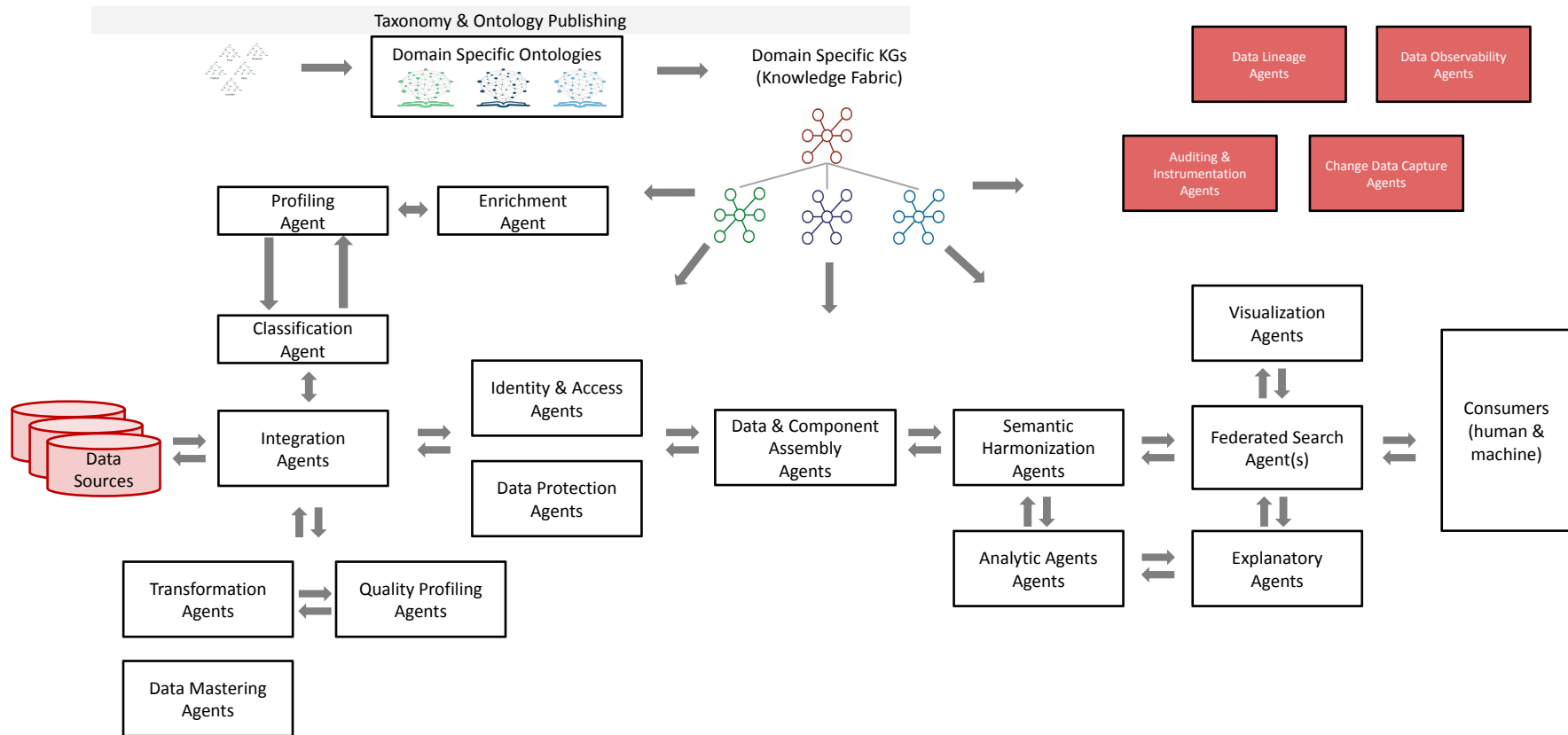
Design Concepts & Considerations

May 2024

Embracing Agent-Based Insights & Governance

Goal is to migrate away from centralized points of functionality and dependencies towards an agent-based architecture with intelligent agents powered by AI, SLMs, and microservices, grounded by domain-specific (semantic) knowledge

Conceptual overview of Agent-Based Insights & Governance



Design Considerations:

Agent-Based Architecture:

- Intelligent Agents that possess cognitive capabilities to perceive, reason, and act autonomously.
- Agent Based Communication provides a messaging protocol for agents to communicate, exchange information, and collaborate in a decentralized manner.
- Lifecycle Management for agent creation, deployment, monitoring, and retirement
- Business Continuity to ensure scalability and fault tolerance is in place during Design Time & Run Time for managing a large number of agents.

Building Intelligent Agents:

- Component Design (reuse) and interoperability with AI, SLMs
- Define domain specific ontologies representing the knowledge and relationships within the domain.
- Ontologies serve as a semantic framework for agents to interpret and reason about events and concepts.
- SLMs trained on domain-specific KGs (NL understanding, contextual reasoning)
- Embedding AI models and SLMs within agents for real-time advanced decision-making, natural language understanding, and contextual reasoning.
- Implement a logic framework (SHAQL within the Semantic Knowledge Layer) to govern the behavior of agents based on domain-specific rules, policies, and constraints.

Event-Driven Architecture:

- Decompose system functionality into loosely coupled microservices
- Transition function points from monolithic tools into discrete microservices that can be executed reliably isolation or as part of a defined flow
- Event-Driven Communication leveraging patterns to enable asynchronous interaction between microservices and agents (for agent actions and decision-making)
- Service Discovery (naming, attribution, registry) and Orchestration facilitating dynamic interaction and composition of microservices by agents

Design Considerations:

Autonomous Decision-Making:

- Event-Driven Processing guided by agent-driven decisions in real-time
- Discrete functions encapsulated by agent specialists for ingesting, processing, and analyzing outputs from events across a variety of sources
- Embed decision-making logic within agents to analyze incoming events, consult domain ontologies, and make context-aware decisions autonomously.
- Support Intra-Domain & Inter-Domain collaboration between agents to share information, coordinate decisions, and orchestrate actions to achieve common goals.

Adaptive Learning and Emergent Behavior:

- Develop models or algorithms to enable emergent behaviors and self-organization among agents based on interactions and environmental stimuli
- Incorporate adaptive learning mechanisms within agents to adapt their behavior, decision-making strategies, and ontology mappings over time based on feedback and environmental changes.
- Establish reinforcement learning techniques to enable agents to learn from experiences and optimize their behavior to achieve desired outcomes.
- Transition scenario-specific events, patterns, and outcomes into Domain-Specific Ontologies to memorialize deterministic knowledge

Governance Controls:

- Extend ACL-based access and ADOM Group mappings into scenario-driven patterns in the Semantic Knowledge Layer
- Implement agent-based provisioning to enforce compliance with security policies
- Implement agent-based protection to mask, encrypt, and control releasability of data based on compliance rules exposed through semantic knowledge layer
- Create auditing & instrumentation agents to mine logs, capture changes and support lineage, observability, auditing, and diagnostic agents
- Create agents to cross-check agent decision-making ensuring adherence to ethical AI principles and guidelines to ensure fairness, accountability, and transparency in agent behavior and decision-making.

Design Considerations:

Scalability and Performance:

- Agents & framework need to be designed for horizontal scale to handle increasing volumes of events, interactions, and agent instances.
- Continuous tuning is a hallmark for performance optimization covering agent communication, event processing, and decision-making algorithms to minimize latency and maximize throughput.
- Workload balancing requires recursive learning with resource management techniques to efficiently utilize computing, memory, and hierarchical storage to ensure optimal performance under varying workloads.

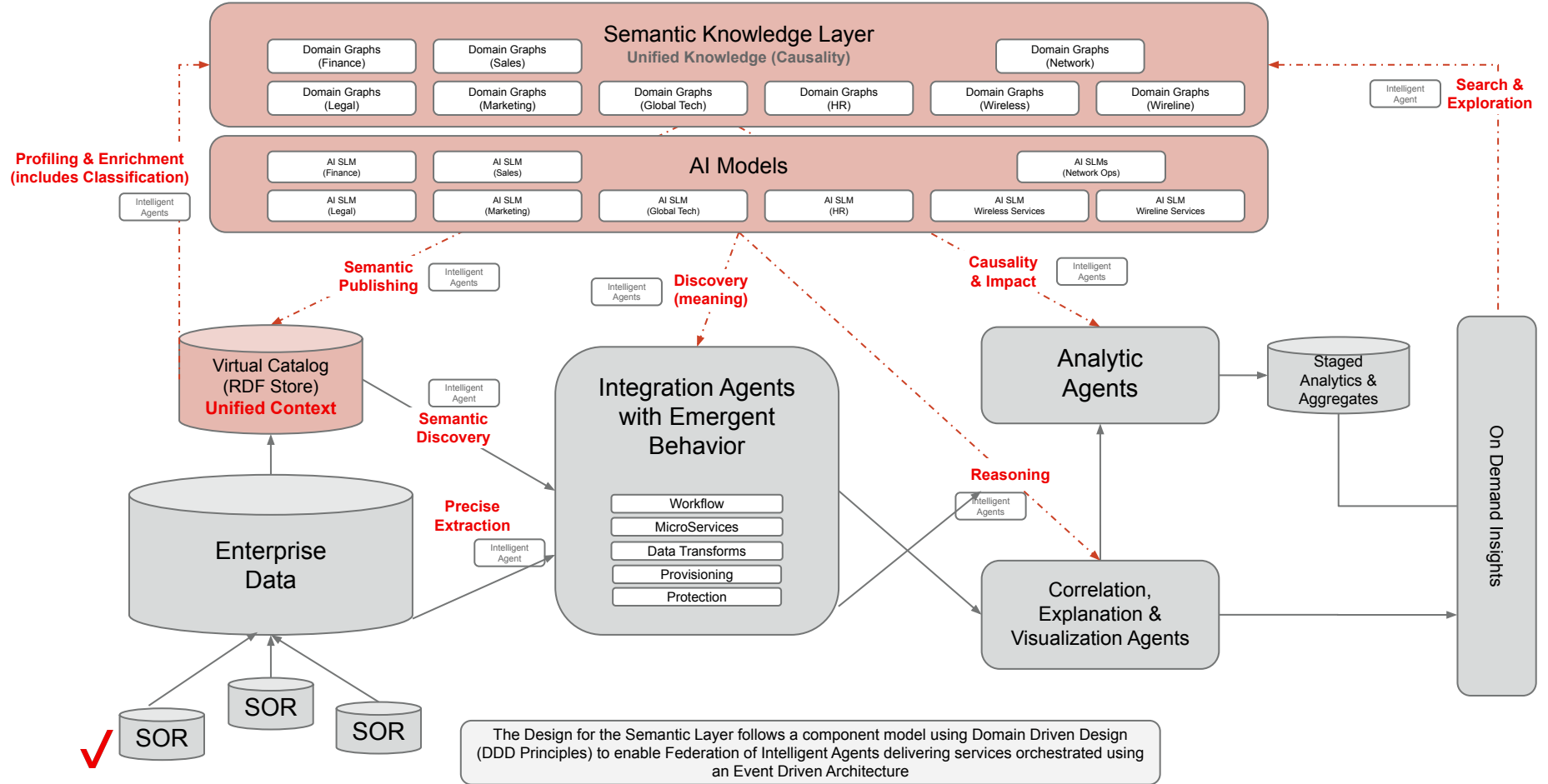
Integration and Interoperability:

- Standardized interfaces across agents, engines, and tools ensuring seamless and resilient integration.
- Design Time & Run Time assurance across agents, systems, sources, and services requires lifecycle discipline supporting 4 9's integrity.
- Design for interoperability is a much higher benchmark than integration to minimize cascading failures and business interruption
- Legacy System Integration will require insulation (special adapters and/or facades). Hard wire connects and simple handoffs will not carry the day within an agent-based architecture (same care must be taken when introducing any new 3rd party components into this organic ecosystem)

Testing and Validation:

- Simulation Environment: Create simulation environments to test agent behavior, interactions, and emergent behaviors in controlled settings.
- Integration Testing: Conduct integration testing to validate the interoperability and compatibility of agents, microservices, and external systems.
- Performance Testing: Perform performance testing to evaluate the scalability, reliability, and responsiveness of the agent-based architecture under varying loads and conditions

Federated Knowledge Driving Emergent & Adaptable Capabilities



The Long Game...

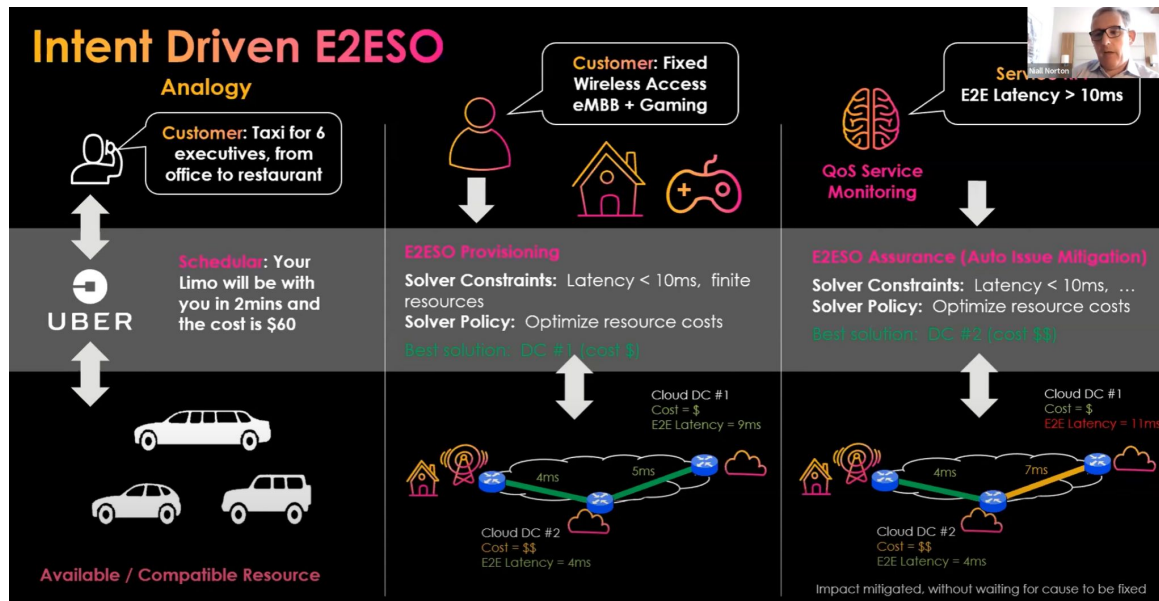
Goal for embarking on this effort is to develop the internal capabilities, apply proven lifecycle rigor across engineering disciplines to build an industrial strength platform to deliver enterprise agility.

We can continue to work with vendors, but they need to play by the new rules that minimize disruption in our operational and strategic goals. Buy vs. Build will shift towards Buy + Insulate and partners that provide services will be held to the 5 9's standard to meet our operational needs.

Once we prove out the “ilities” for our internal customers, we can begin externalize these agent-based services with partners to create new value streams and eventually participate in value chains for customers.

Leveraging Knowledge to Accelerate Telco Operation

Service Automation & Orchestration in Telecom



Supporting Materials

Enabling Value Chains with Agentic Design

Today, we always think about our technology in terms of solutions not necessarily services within a larger Value Stream (internally) and Value Chains (externally).

As we move forward, regulating bodies need the ability to trace & understand how AI models are consumed and reused.

The additional responsibilities as a Developer vs Deployer also come into play based on the type changes we make to an output along the way. This can get complicated as we step through the chain-of-delivery / custody.

Here's a simple outline that we may want to discuss further as a consistent framework from which to measure value | quality | trust | compliance | impact.

Value Chain

Value Stream(s)

Service Chain(s)

Process(s)

Input*

Processing*

Decision Logic

Discrete Activities

Output*

Measurement**

Measurement

Measurement

Measurement

Measurement

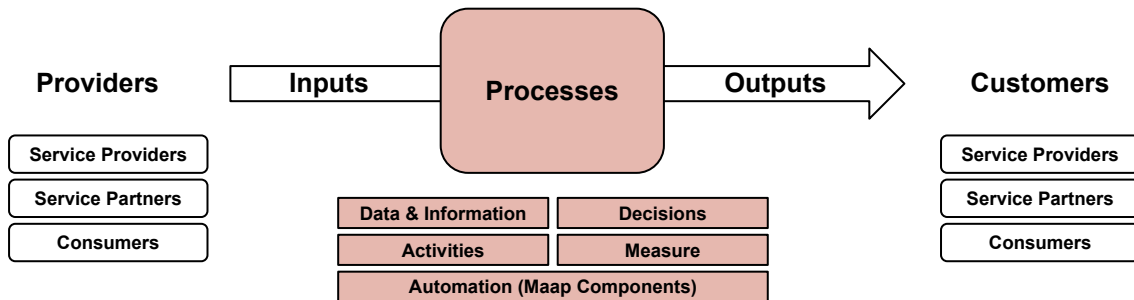
Controls

*Prevent

**Detect

Service Design: “Mini-Supply Chain”

What is a Service?
(Think about it as a “mini-Supply Chain”)



A Service (Business, Functional, Technical):

- Capabilities are enabled by one or more services
- Each service is designed as a “mini-supply chain” (SIPOCR - see examples below)
- Service Level Management is designed to ensure operational effectiveness
- Simplify accountability (Service Matrix is consistent versus RACI Models which always change)
- Measures quality, productivity, and service excellence (what gets measured Right gets done Right)
- Service is a business model (revenue, cost control, measurement, supplier and customer management)
- Service owners are responsible for delivering desired outcomes for customers, compliance, & corporate

When we “connect-the-dots” between Service Owners we can create Value Streams (“legos” of Reusable Component Services)

This allows us to quickly bootstrap (new) Value-Added Services like Data Onboarding

A few examples of Service Handoffs between Service Owners (manual & automated)

Supplier (Provider)	Input	Service Owner	Process (Service - L2 & L3)	Output	Customer (Consumer)
Consumers Compliance Corporate	Jobs + Desired Outcomes	Product Management	Design Thinking (Outcome Discovery)	Personas & Job Study	Data Architecture Data Engineering Data Governance
Catalog Management	Updated Business Term Mapping to Policy Terms	Orchestration Services	Replicate Changes to Enterprise Data Catalog to GCP Data Catalog	CSV File (Column to Policy Term Mapping)	System Operations GCP (DGF) GCP (VCF)

Classification of Agents...

This needs a lot more discussion. There's a movement to associate agents with personas to mimic expertise associated with Service Providers and Service Partners.

There's also an opportunity to group agents by common function points:

- React (reflex) Agents
 - Evaluates (sensors) to identify impacts of changing conditions relative to a baseline state
 - May present options for responding (actuators)
- Goal-Based Agents
 - Evaluates options to move an existing state to a future (desired) state
 - Provides what-if scenario evaluation (limited) based on one or more options (actions) to progress towards future state
- Utility Agents
 - These are well defined actions that are performed and precisely measured to ensure they are performed consistently and reliably within a bounded set of parameters.
- Learning Agents
 - Capturing and curating knowledge relative to performance. Deviations from performance and issues are captured along with the relevant context to identify future occurrences within a bounded understanding of scenarios

Then, there's the mapping of concerns: Personas, Use Cases, Agents...

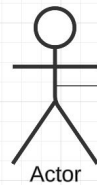
Need to understand granularity & cardinality in the larger scheme:

- Actor (= Persona)
 - 1:M Use Case (taxa?)
 - 1:M Intents
 - 1:M Services
- Service
 - Integration
 - Agent (Bi)
 - Agent (AI)
 - Agent (DI)
- Service granularity (taxa?)
- Agent granularity (taxa?)

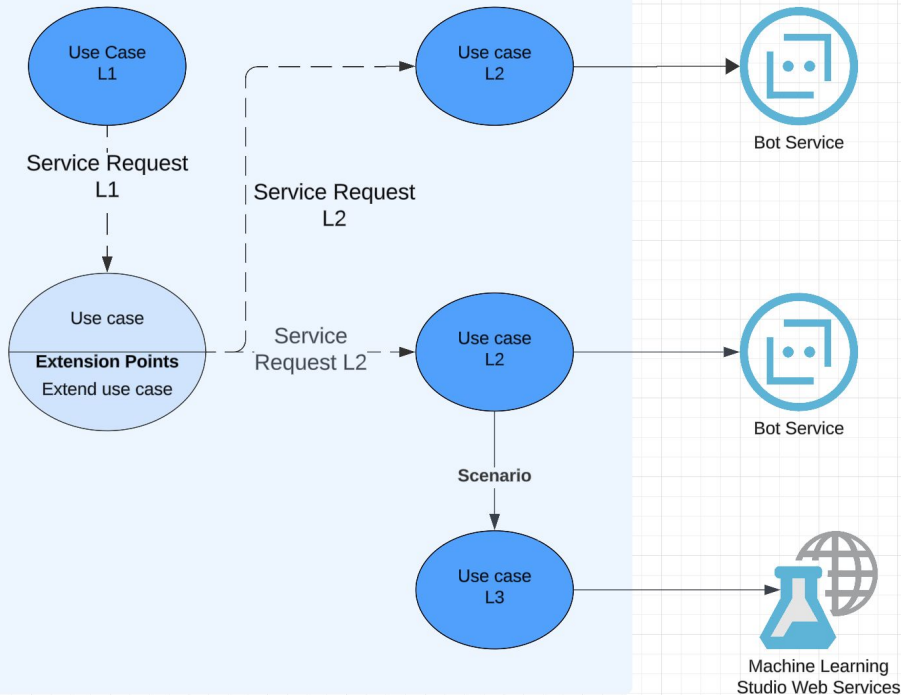
We may need to build registries covering:

- Use Cases
- Services
- Agents
- Models
- Metrics
- Decisions

Need a scalable way to track usage (embed) and/or consumption (service requests). Former is Design Time, Latter is Run Time



Assembly (UCs & Services with AI & GenAi)



Lifecycle Considerations - Blending of Design Time & Run Time

As we step into Agent-Based design, we may want to align on some target state capabilities so we reduce the costs for refactoring agents. Much of our solution design is still macro level focus as design is driven by Use Cases, which are also macro level and cross-domain concerns.

Do we need to start thinking about

== > **Lifecycle Design for Build & Assembly (static, then dynamic)**

Lifecycle Considerations - DFMA as a Discipline

DFMA = Design for Manufacturing and Assembly (introduced to support Industry 4.0+)

This has been around for a number of years and is gaining momentum as companies begin exploring agent based design.

DFMA is the combination of DFM (mfg) and DFA (assembly)

- Focus on simplifying product with manufacturing ease and efficient assembly.
- Minimum Time (Shorter time to market)
- Lower cost
- Increased reliability

Core Principles:

- Make multi functioned parts
- Minimize parts
- Simplify/optimize manufacturing process.
- Eliminate interfaces
- Use standard components
- Ease in handling
- Minimize reorientation during machining or assembly
- Encourages concurrent engineering
- Take advantage of modular assembly

Lifecycle Considerations - Learning in Complex Systems

DFA and Emergent Behavior Theory maps very nicely into our goals with AI, BI, SI (semantic), and DI (decision):

- Collective Learning (top level domain)
- Collaborative Learning (inter-domain)
- Organized Learning (intra-domain = bottom up)
- Organizational Learning (macro)

Some of the challenges we are facing with complex systems involve computational resource optimization, linguistic translation, and algorithmic complexity when we REACT using our solution designs